



**SAPIENZA**  
UNIVERSITÀ DI ROMA

DEPARTMENT OF COMPUTER SCIENCE

# **From Models to Suites: FSM-Based Robot Test Generator**

ADVANCED SOFTWARE ENGINEERING

**Professors:**

Paolo Bottoni

**Students:**

Lucian Dorin Crainic

1938430

---

Academic Year 2024/2025

## **Abstract**

This document presents the research conducted on FSM-based test generation for robotic systems. The focus is on the methodologies employed, the results obtained, and the implications for future research in automated testing.

# Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>3</b>  |
| 1.1      | Motivation: Why FSM-based Testing? . . . . .               | 3         |
| 1.2      | Project objective and scope . . . . .                      | 3         |
| 1.3      | Main contributions . . . . .                               | 3         |
| <b>2</b> | <b>Background and Related Work</b>                         | <b>4</b>  |
| 2.1      | Finite-State Machines (FSMs) for Test Generation . . . . . | 4         |
| 2.2      | Robot Framework Essentials . . . . .                       | 5         |
| 2.3      | Coverage Criteria (State, Transition, Action) . . . . .    | 6         |
| 2.4      | Comparable Tools and Approaches . . . . .                  | 6         |
| <b>3</b> | <b>FSM Grammar Specification (.machine)</b>                | <b>8</b>  |
| 3.1      | Design goals . . . . .                                     | 8         |
| 3.2      | Complete EBNF grammar . . . . .                            | 8         |
| 3.3      | Semantic validation rules . . . . .                        | 8         |
| <b>4</b> | <b>Generator Design and Implementation</b>                 | <b>9</b>  |
| 4.1      | High-Level Design . . . . .                                | 9         |
| 4.2      | Main data flow . . . . .                                   | 9         |
| 4.3      | Strategy Interface and Implementations . . . . .           | 9         |
| 4.4      | Core Implementation highlights . . . . .                   | 9         |
| <b>5</b> | <b>Verification and Validation</b>                         | <b>10</b> |
| 5.1      | Unit Tests . . . . .                                       | 10        |
| 5.2      | Integration Tests . . . . .                                | 10        |
| 5.3      | Login FSM example . . . . .                                | 10        |
| <b>6</b> | <b>Real-project case study</b>                             | <b>11</b> |
| 6.1      | System description . . . . .                               | 11        |
| 6.2      | Typical workflow in practice . . . . .                     | 11        |
| 6.3      | Different test generation results . . . . .                | 11        |
| 6.4      | Discussion and future work . . . . .                       | 11        |
|          | <b>References</b>                                          | <b>12</b> |
|          | <b>References</b>                                          | <b>12</b> |

# **1 Introduction**

## **1.1 Motivation: Why FSM-based Testing?**

## **1.2 Project objective and scope**

## **1.3 Main contributions**

## 2 Background and Related Work

### 2.1 Finite-State Machines (FSMs) for Test Generation

Finite-state machines (FSMs) describe software behavior as a collection of states and the transitions that connect them. Each transition is triggered by an input and may produce an output, making FSMs well suited for modelling reactive systems[1]. Test generation from an FSM typically begins with a formal specification that enumerates all possible states of the system under consideration. From this specification, automated tools derive sequences of inputs that traverse the state space. The resulting test cases explore systematic paths through the model and help uncover defects that manifest only after a particular series of events.

Early research established a direct link between FSM models and systematic test coverage. Chow[2] introduced the W-method to generate tests that exercise every transition of a minimal deterministic machine. Subsequent work extended these concepts to non-deterministic and partially specified models. A common pattern among these methods is to treat the FSM as a graph and search for paths that satisfy specific coverage goals. For instance, breadth-first search can be used to create a test suite that visits every state at least once, while more advanced techniques may incorporate heuristics for minimising the number of steps.

Figure 1 illustrates a small FSM representing a user login workflow. The model identifies valid credentials, incorrect attempts, and the lockout state after successive failures. Although the diagram is simplistic, it highlights how model-based tests can capture user interactions with strict control over the order of events. Complex systems typically contain many more states, and exhaustive enumeration quickly becomes impractical. Therefore, researchers have proposed abstraction and compositional reasoning to keep the models tractable. Such approaches allow testers to focus on critical regions of the state space without enumerating every permutation.

Figure 1: Example finite-state machine representing a simplified login process.

In modern development workflows, FSM-based test generation forms the backbone of many model-based testing (MBT) tools. These tools integrate with continuous integration pipelines to execute automatically derived tests whenever the model or system under test changes. Despite progress in automation, practitioners must still craft models carefully to balance completeness and maintainability. The remainder of this chapter discusses how FSMs integrate with the Robot Framework and how various coverage criteria ensure the generated tests provide meaningful assurance.

## 2.2 Robot Framework Essentials

The Robot Framework is an open-source automation platform designed for acceptance and end-to-end testing. Tests in Robot Framework are written in a tabular syntax, which emphasises readability and encourages collaboration between developers and nontechnical stakeholders[3]. At the core of Robot Framework is a keyword-driven approach, where keywords represent high-level actions or verifications. Built-in and external libraries provide keywords for interacting with web browsers, databases, application interfaces, and more.

Robot Framework test suites follow a consistent structure consisting of settings, variables, test cases, and optional keywords. Settings declare resources such as libraries, resource files, and metadata. Variables simplify parameterisation by making values reusable across the suite. The test cases themselves describe a series of keywords executed in order. By defining custom keywords, teams can encapsulate complex actions and create a domain-specific language tailored to their application.

Listing 1 shows a minimal Robot Framework test that verifies a successful login. The script opens a browser, enters credentials, and checks for a welcoming element on the resulting page. The example uses the SeleniumLibrary, one of the most widely adopted libraries for web testing. While simple, the listing demonstrates how a few lines in Robot syntax orchestrate a complete test scenario.

```
1 *** Settings ***
2 Library           SeleniumLibrary
3
4 *** Test Cases ***
5 Valid Login
6     Open Browser    ${LOGIN_URL}    chrome
7     Input Text      id=username     demo
8     Input Text      id=password     secret
9     Click Button    id=login
10    Page Should Contain Element    css=.welcome
11    [Teardown]        Close Browser
```

Listing 1: Simple login test in Robot Framework

Robot Framework supports a rich ecosystem of plugins and tools. Tags can be used for selective execution, and listeners provide hooks for custom reporting. The framework integrates with continuous integration servers and allows tests to be written in plain text, reStructuredText, or Markdown formats. Because of its extensibility, Robot Framework is an ideal host environment for model-based test generation, where models can be translated into high-level keywords that map directly to system interactions.

## 2.3 Coverage Criteria (State, Transition, Action)

Coverage metrics help quantify how thoroughly a set of test cases exercises the model of a system. For FSM-based approaches, three fundamental criteria are commonly considered: state coverage, transition coverage, and action coverage. State coverage requires every reachable state in the model to appear in at least one test case. Achieving full state coverage ensures the test suite observes the system in all of its documented configurations, though it does not guarantee that every change of state is exercised.

Transition coverage goes a step further by insisting that each directed edge in the FSM be executed by some test. This criterion verifies that the system under investigation handles all allowable inputs at every decision point. Various algorithms exist to derive a minimal set of paths that collectively cover every transition[4]. In practice, complete transition coverage may be unreachable for complex or nondeterministic systems, so engineers often focus on critical subsets of transitions.

Action coverage pertains to the operations or outputs associated with transitions. An action might correspond to a method invocation, a message sent on a queue, or an update to persistent state. Action coverage requires that each unique action defined in the FSM is observed during execution. This criterion is useful when transitions share common labels or when additional side effects need verification beyond state changes. Some tools track action coverage by instrumenting the test harness or by parsing logs to associate actions with model elements.

Existing versions of this project provide basic state and transition tracking, but finer-grained action coverage is earmarked for future work. Integrating coverage tooling directly into the Robot Framework environment will allow continuous measurement as tests execute, providing immediate feedback on which parts of the model remain untested.

## 2.4 Comparable Tools and Approaches

Multiple model-based testing tools employ FSMs or related formalisms to generate test cases. GraphWalker is an open-source tool that interprets models described in a graph notation and executes random or guided walks to produce test sequences[5]. It integrates with Java and allows assertions to be placed on edges or vertices. Modbat is another approach, offering a Scala-based environment for modelling probabilistic automata[6]. Modbat can express complex concurrency scenarios and uses randomised exploration combined with user-defined assertions.

Tcases focuses on input space modelling rather than explicit state machines. It employs combinatorial techniques to derive minimal test sets that cover interactions among input parameters[7]. Although Tcases does not rely on an FSM, it shares the goal of systematic coverage and can complement state-based approaches. Other tools, such as QuickCheck and its derivatives, utilise property-based testing to generate large

numbers of input sequences that satisfy high-level properties.

| Tool        | Model Type              | Notable Feature              |
|-------------|-------------------------|------------------------------|
| GraphWalker | Graph-based FSM         | Guided and random traversals |
| Modbat      | Probabilistic automata  | Concurrency support          |
| Tcases      | Input parameter model   | Combinatorial coverage       |
| QuickCheck  | Property specifications | Randomised test data         |

Table 1: Representative tools for model-based test generation.

Compared with these tools, the present project emphasises ease of integration with the Robot Framework and straightforward model specification. While GraphWalker and Modbat target Java or Scala environments, our approach focuses on Python-based tooling and the readable syntax favoured by Robot Framework users. Tcases and QuickCheck address different kinds of models, yet they share complementary goals of systematic exploration. The methodology outlined in this work seeks to combine the accessibility of Robot Framework with the thoroughness of MBT techniques. The following chapter introduces the concrete design and implementation choices made to realise this integration.



## **3   FSM Grammar Specification (.machine)**

### **3.1   Design goals**

### **3.2   Complete EBNF grammar**

### **3.3   Semantic validation rules**

## 4 Generator Design and Implementation

### 4.1 High-Level Design

### 4.2 Main data flow

### 4.3 Strategy Interface and Implementations

### 4.4 Core Implementation highlights

## 5 Verification and Validation

### 5.1 Unit Tests

### 5.2 Integration Tests

### 5.3 Login FSM example

## 6 Real-project case study

### 6.1 System description

### 6.2 Typical workflow in practice

### 6.3 Different test generation results

### 6.4 Discussion and future work

## References

- [1] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 2001.
- [2] T. S. Chow. Testing software design modeled by finite-state machines. *IEEE Transactions on Software Engineering*, SE-4(3):178–187, 1978.
- [3] Robot Framework Foundation. Robot framework user guide. <https://robotframework.org/>. Accessed: 2025-07-13.
- [4] Robert V. Binder. *Testing Object-Oriented Systems: Models, Patterns, and Tools*. Addison-Wesley, 1999.
- [5] GraphWalker Project. Graphwalker. <https://graphwalker.github.io/>. Accessed: 2025-07-13.
- [6] Matthias Heizmann. Modbat. <https://github.com/codelabs-ch/modbat>. Accessed: 2025-07-13.
- [7] Kerry Cornutum. Tcases. <https://github.com/Cornutum/tcases>. Accessed: 2025-07-13.